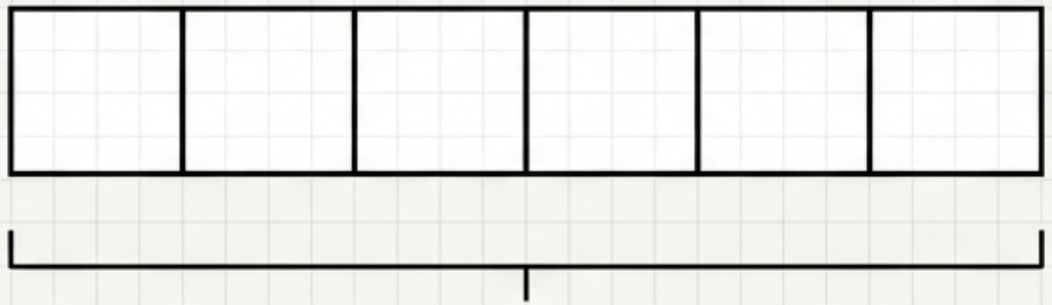
Selection Sort: The Concept-to-Code Blueprint

A visual execution guide to algorithmic ordering.

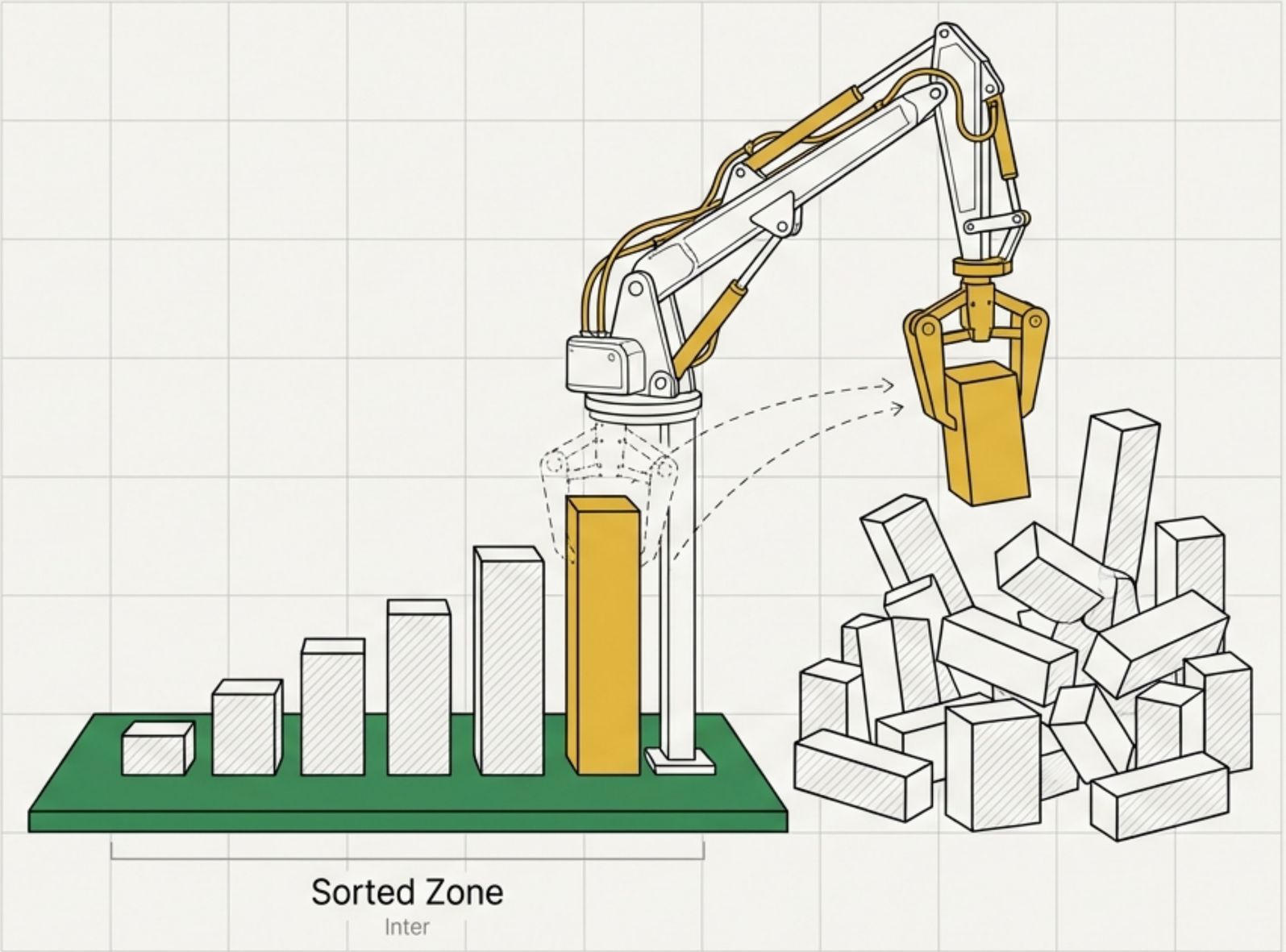


STATUS: UNSORTED

Order Out of Chaos

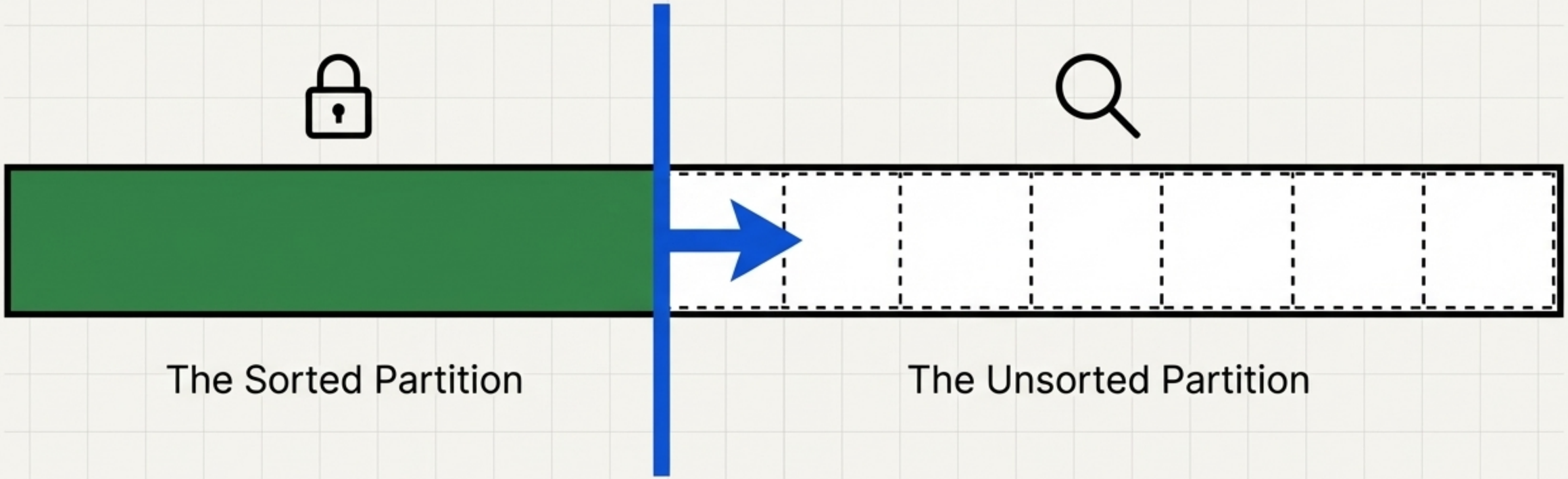
The premise of Selection Sort is absolute certainty. Instead of shuffling elements blindly, it repeatedly scans the unsorted data, selects the absolute minimum value, and permanently locks it into its definitive final position.

Key Insight: Every iteration guarantees one completely finalized element. The sorted zone only grows; it never shrinks.

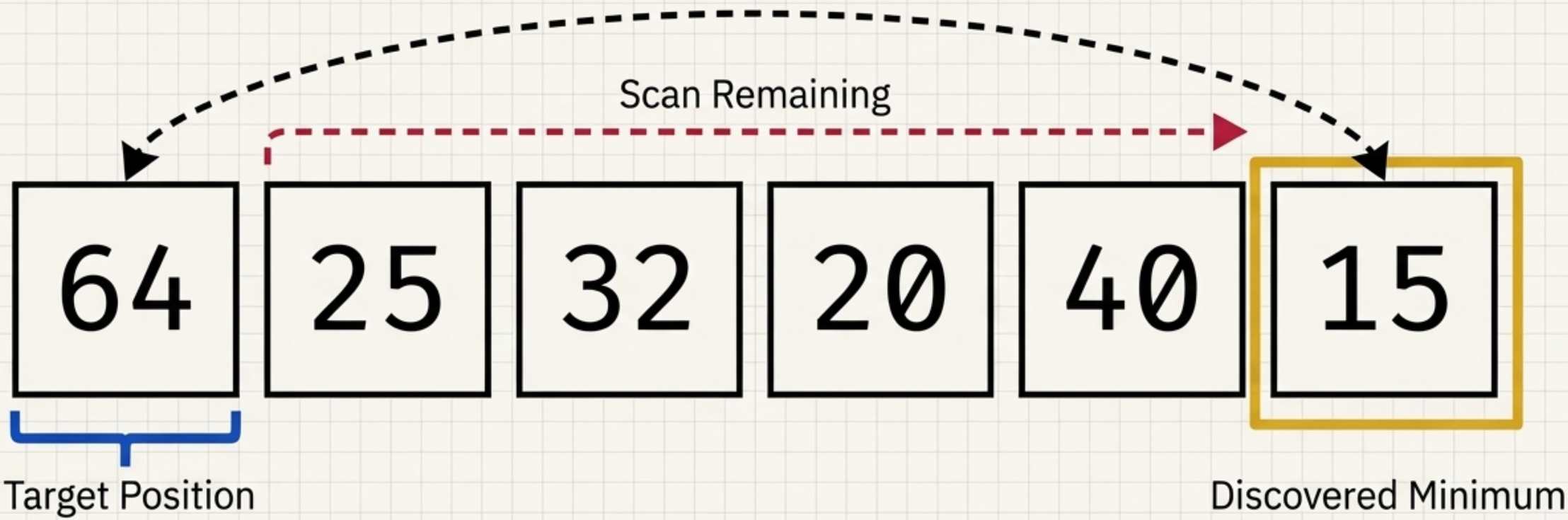


The Shifting Boundary

At any given moment, the array exists in two states simultaneously. The algorithm's only job is to move the boundary one position to the right per iteration.



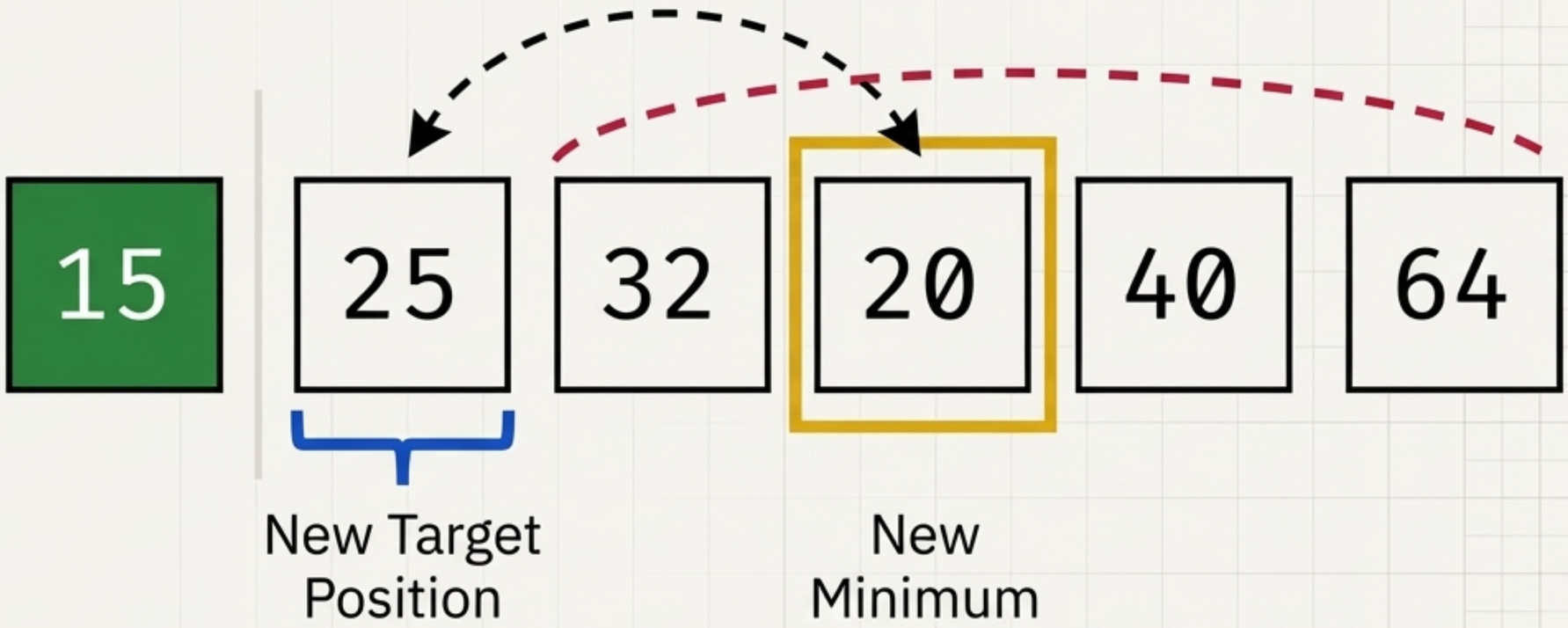
State 0: The First Pass



1. Assume the first element (64) is the minimum.
2. Scan the rest of the array.
3. Discover the true minimum (15).
4. Swap them.

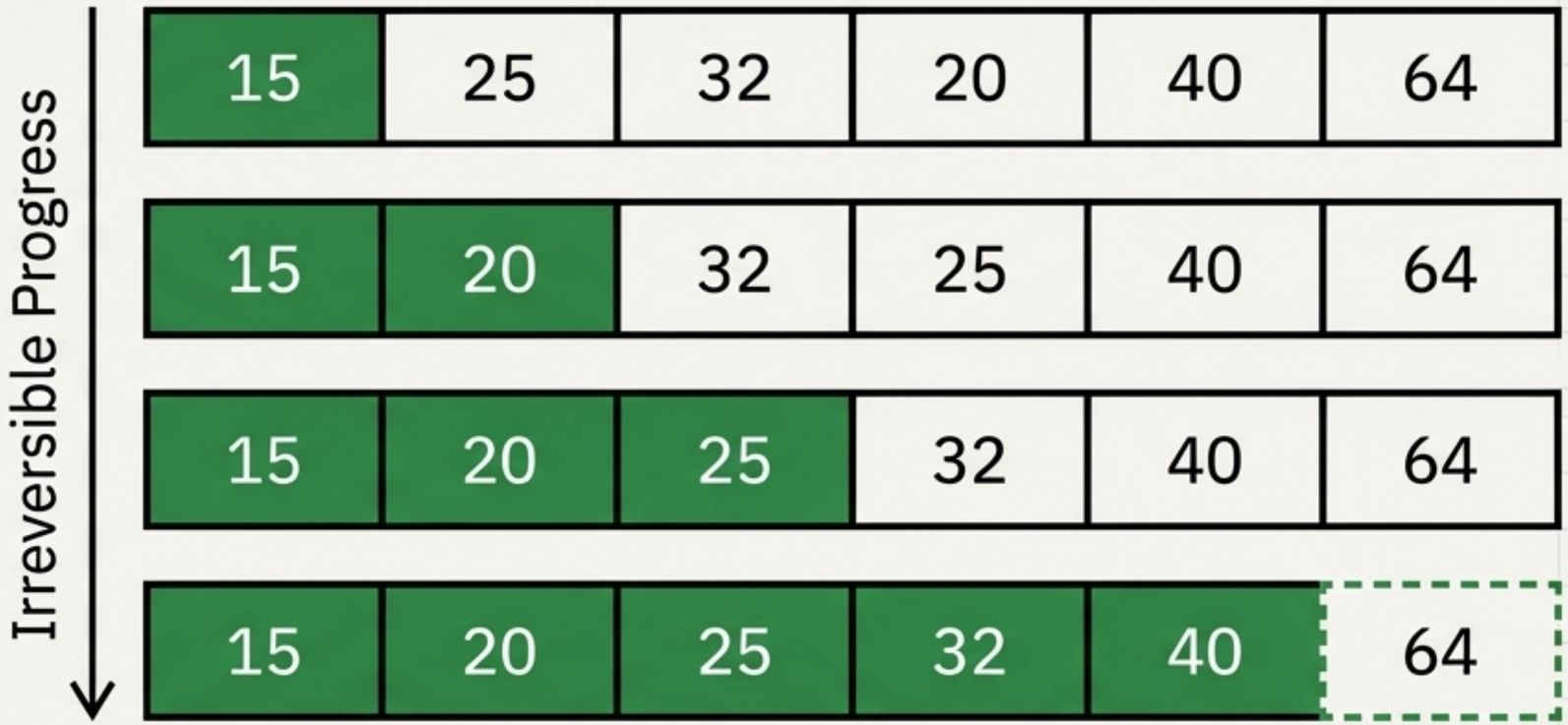
Iteration 1: Expanding the Boundary

The number 15 is locked and ignored. The boundary moves right. The new target is index 1, but the scan reveals 20 as the true minimum of the remaining pile.

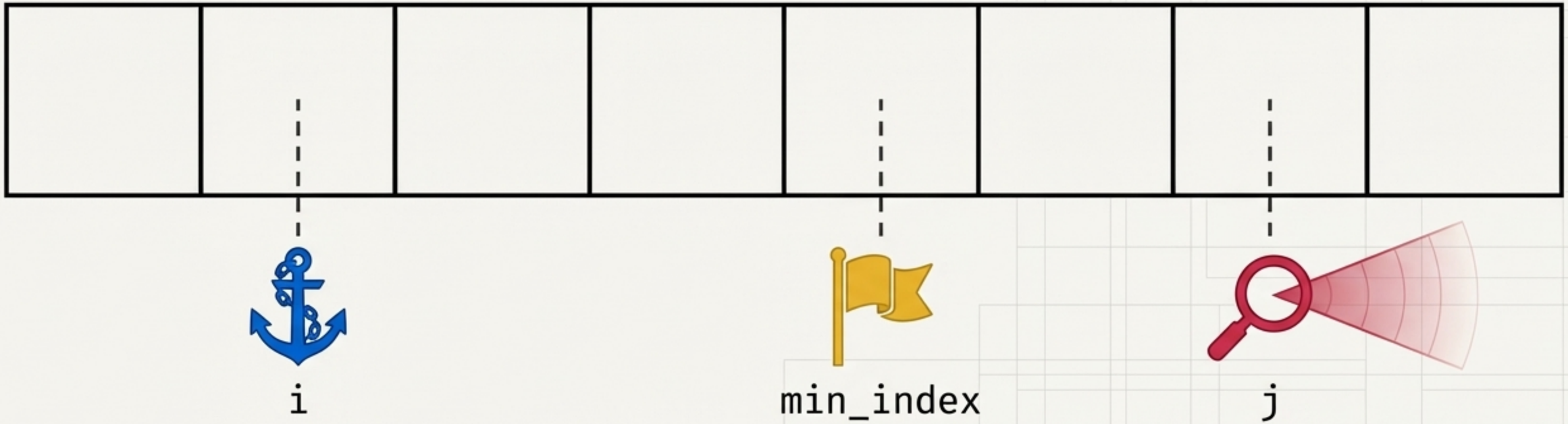


The Cascade of Order

With each full scan, the sorted zone grows by exactly one element. By the time the boundary reaches the final element, the entire system is intrinsically sorted.



The Architecture of Search



The Anchor (i)

Defines the current target position. Marks the precise edge of the sorted zone.

The Scout (j)

Deploys from the anchor's position. Scans forward iteratively to the very end of the array.

The Flag (min_index)

A temporary marker dropped by the scout whenever it finds a new smallest number.

System Constraints: Outer vs. Inner

Dimension	Outer Loop (i)	Inner Loop (j)
Identity	🔵 The Anchor	🔴 The Scout
Origin (Start Point)	`0` (Moves forward by 1 each pass)	`i` (Always starts at the current Anchor)
Terminus (End Point)	n (End of the array)	n (End of the array)
Primary Objective	Dictates WHERE the next element goes	Discovers WHAT the next element is

Translating Intuition into Syntax

Human Logic	Computer Logic
Let's assume the first unsorted item is the smallest.	`min_index = i`
Look at every single remaining item one by one.	`for j in range(i, n):`
If we see something even smaller, update our assumption.	`if a[j] < a[min_index]:` `  min_index = j`

The Complete Blueprint

```
n = len(a)
for i in range(n):
    min = i
    for j in range(i, n):
        if a[j] < a[min]:
            min = j
    a[i], a[min] = a[min], a[i]
```

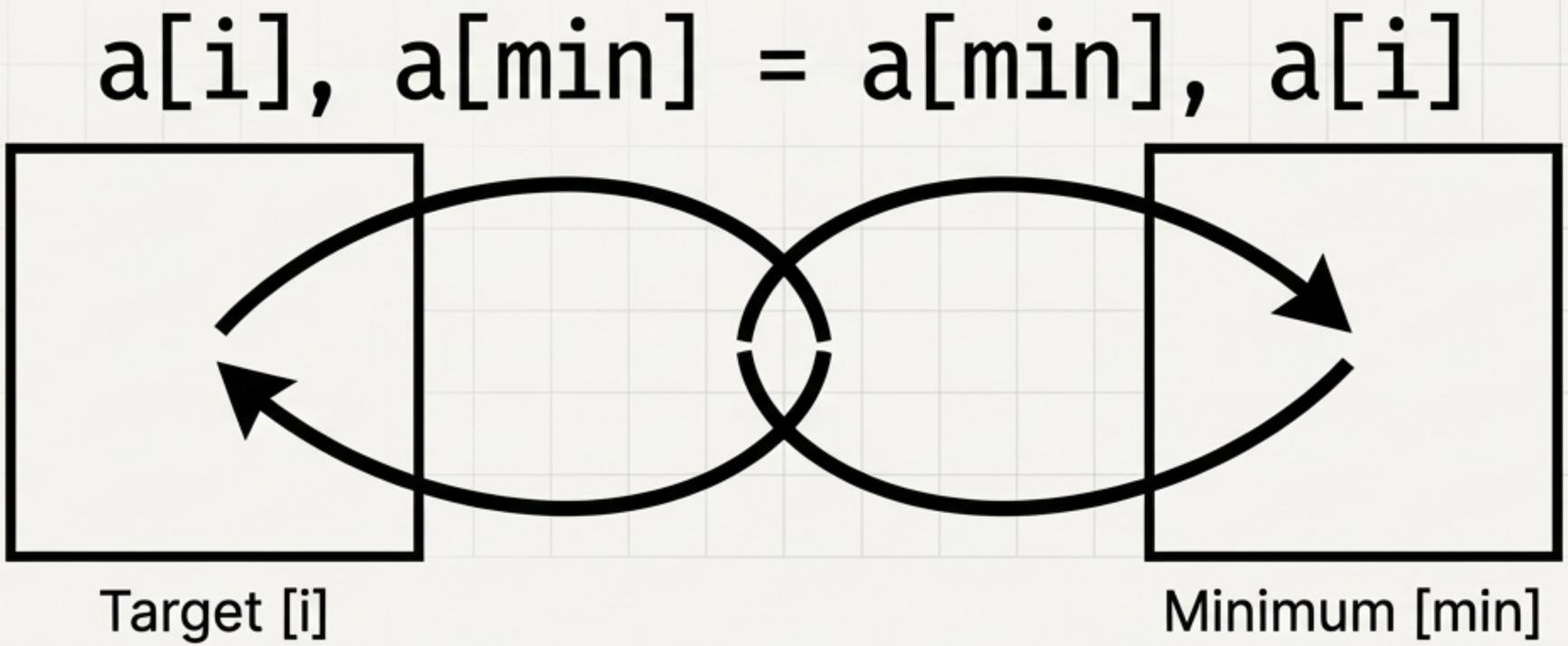
The Anchor's loop

The Scout's scan

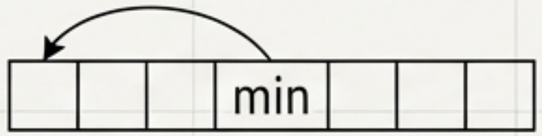
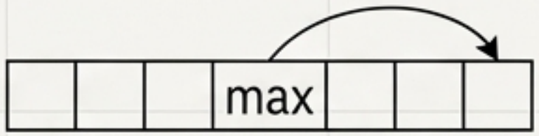
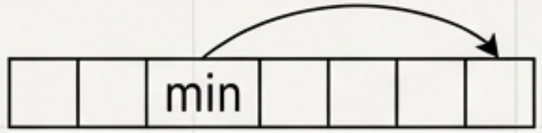
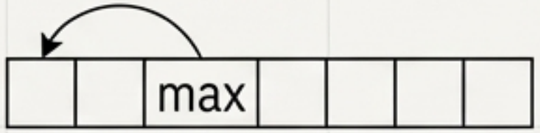
The Boundary Swap

The Pythonic Swap

Unlike other languages that require a temporary holding variable, Python handles the exchange natively through tuple unpacking. The target location and the discovered minimum location exchange their values instantly.



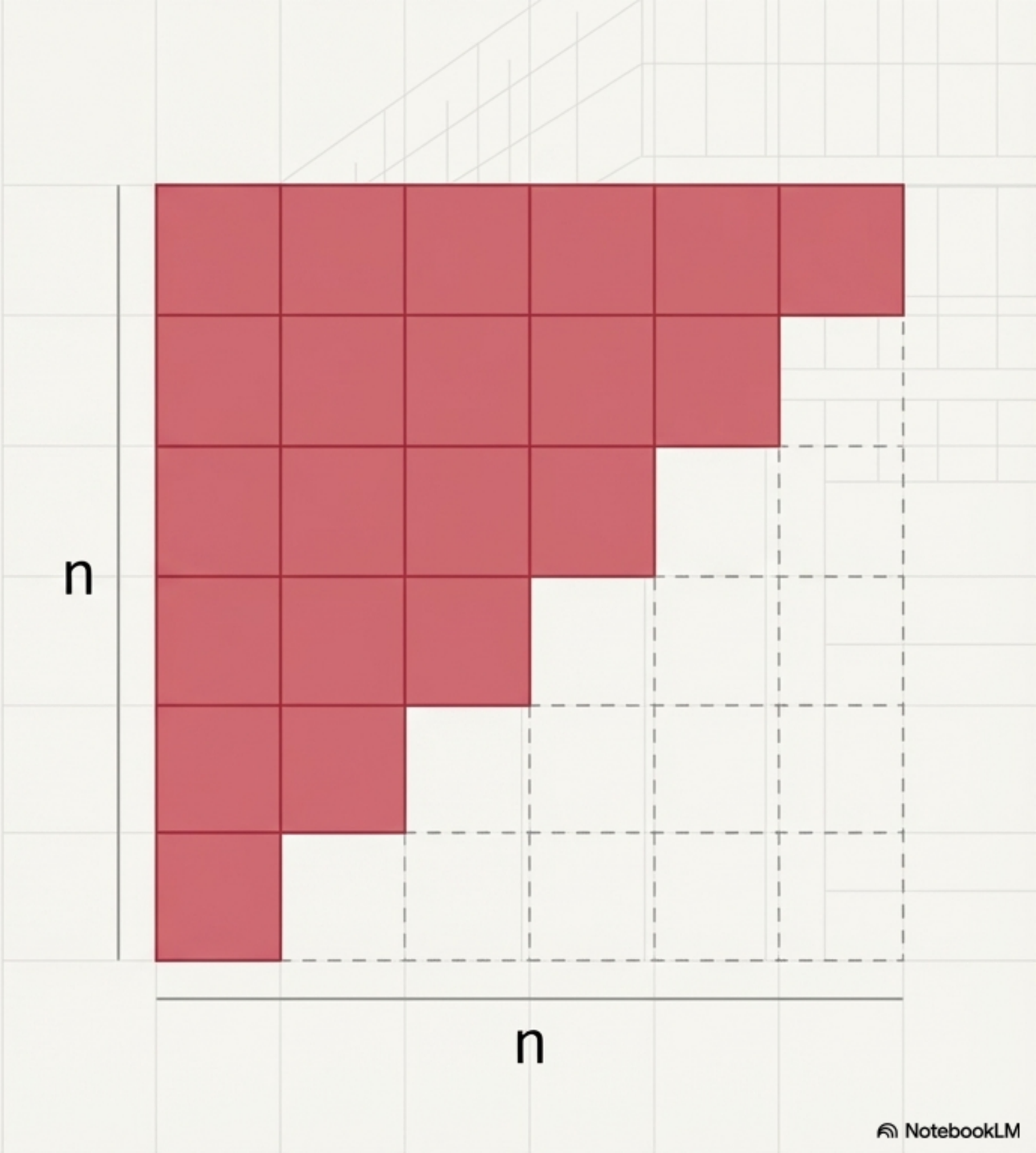
Order Variations & Flexibility

	The Min Approach	The Max Approach
Increasing Order	Find Minimum -> Swap to Front 	Find Maximum -> Swap to Back 
Decreasing Order	Find Minimum -> Swap to Back 	Find Maximum -> Swap to Front 

The core loop structure remains identical. Only the target extreme and the anchor position change.

The Cost of Certainty: $O(n^2)$

The outer loop runs n times. The inner loop initially runs n times, then n-1, then n-2. Plotted out, the number of comparisons forms a triangle representing roughly $n^2/2$ operations. Because both loops are strictly dependent on the data size, the time complexity firmly resolves to $O(n^2)$.



The Definitive Truth of Selection Sort

It is deeply systematic. It minimizes the physical moving of data by performing exhaustive cognitive work before making a single swap. Find the extreme. Lock it in place. Repeat.